Working with EFS

Explore how to mount and manage Amazon Elastic File System (EFS) with EC2 instances and AWS Lambda functions. Understand EFS setup, security, and monitoring to build scalable, low-latency storage solutions suitable for distributed applications and persistent function storage in your AWS environment.

Amazon Elastic File System (EFS) offers fully managed, scalable, and elastic file storage that integrates deeply with other AWS compute services, especially Amazon EC2 and AWS Lambda. By mounting EFS to EC2 instances or connecting it to Lambda functions, we can provide multiple resources with simultaneous, low-latency access to a consistent file system. This integration is essential for building distributed applications that require shared storage, such as web server clusters, analytics pipelines, or workflows handling large files.

In this lesson, we'll explore how EFS works with EC2, the step-by-step process of mounting EFS, security considerations, integration with Lambda for persistent function storage, and best practices for monitoring and optimizing EFS in production environments.

How Amazon EFS works with EC2

Amazon EC2 and Amazon EFS work together seamlessly to provide shared storage for scalable, distributed applications. When we mount an EFS file system to one or more EC2 instances, those instances can read and write to the same file system concurrently. This enables scenarios like web server clusters, cent logging, content management systems, and any application requiring a consistent file system state across compute nodes.

Ask

EFS is exposed to EC2 instances using the NFSv4.1 or NFSv4.2 protocol. The only requirement is that the EC2 instance be within the same VPC or have appropriate routing to reach the EFS mount targets.

> **Tip:** Mount targets are regional, and AWS recommends placing one in each Availability Zone for high availability and lower latency.

Mounting Amazon EFS on Linux-based EC2 instances

Start by making sure our EC2 instance is in the same VPC as the EFS mount target, or has appropriate network connectivity (via VPC peering or Transit Gateway if needed). Also, ensure that the instance's security group allows outbound access to the EFS mount target's IP and that the EFS mount target's security group allows inbound NFS traffic (port 2049).

Here's how we can mount EFS on a Linux-based EC2 instance in a few simple steps:

1. **Install the Amazon EFS mount helper:** First, let's install the Amazon EFS mount helper. This utility simplifies mounting and provides support for IAM-based access and in-transit encryption.

main.sh Shell

```shell
sudo yum install -y amazon-efs-utils   # For Amazon Linux
# OR
sudo apt install -y amazon-efs-utils   # For Ubuntu/Debian
```

Ask

2. **Create a mount point:** Next, create the directory that will serve as the EFS mount point.

main.sh Shell

```
sudo mkdir -p /mnt/efs
```

3. **Mount the file system:** Now, mount the EFS file system to the previously created mount point.

main.sh Shell

```
sudo mount -t efs <file_system_id>:/ /mnt/efs
```

Ask

To enable encryption of data in transit, include the `-o tls` option:

main.sh Shell

```shell
sudo mount -t efs -o tls <file_system_id>:/ /mnt/efs
```

4. **Persist the mount across reboots:** For automatic mounting of the EFS file system upon system restart, append the following line to `/etc/fstab`. The `_netdev` option ensures the network is available before attempting the mount, and `tls` enables encryption in transit.

main.sh Shell

```shell
<file_system_id>:/ /mnt/efs efs _netdev,tls 0 0
```

Ask

Once mounted, the EFS file system behaves like any other Linux file system—applications can create, read, and modify files without needing to know they're accessing a networked file system.

EC2 instances in different availability zones using local mounting targets to mount EFS

Ask

EFS integration with AWS Lambda

By default, AWS Lambda provides `512 MB` of ephemeral storage in the `/tmp` directory. While sufficient for many tasks, this limitation can be a barrier for developers working with large files, machine learning models, video processing, or shared data workflows. To solve this, AWS allows Lambda functions to mount Amazon EFS file systems, offering scalable, low-latency, and persistent storage that functions can access across invocations and instances.

With this integration, a Lambda function can handle workloads previously constrained by storage limits. We gain persistent storage across invocations, shared access between multiple Lambda functions (or between Lambda and EC2), and the ability to work with large files using standard POSIX-compliant file operations like `open()`, `read()`, `write()`, and `seek()`. The storage behaves like a traditional Linux file system, meaning we don't need to adapt our logic to object-based APIs as one can with S3. The illustration below demonstrates an EFS connected to Lambda and EC2 instances:

Ask

EC2 instances and Lambda function accessing EFS

Let’s look at the steps involved in integrating Lambda with EFS:

Step 1: Create an EFS

Once the EFS is created, we need to set up a mounting target for the targeted service. Each mount target lets resources in a specific subnet access the EFS file system using NFS. For EC2, the mount target simply needs to be reachable. For Lambda, the function must run in the same VPC as the mou target and access it via an access point. The `create-mount-target` CLI command can be used to create a mounting target:

Ask

main.sh Shell

```shell
aws efs create-mount-target \
  --file-system-id <file_system_id> \
  --subnet-id <subnet_id> \
  --security-groups <security_group_id>
```

Create a mount target

Here's a brief explanation of the command:

- `--file-system-id`: It specifies which EFS file system to associate with this mount target.
- `--subnet-id`: It determines which subnet (and therefore, which AZ) the mount target will reside in. For best results, create one mount target per AZ where our compute resources live.
- `--security-groups`: It is the security group that controls inbound/outbound access. It must allow inbound TCP on port `2049` (used by NFS).

Step 2: Create an EFS access point

The next step is to set up an EFS access point. We can use the `create-access-point` command to create an access point:

Ask

main.sh Shell

```shell
aws efs create-access-point \
  --file-system-id <file_system_id> \
```

```
    --posix-user Uid=1000,Gid=1000 \
    --root-directory 'Path=/lambda,CreationInfo={OwnerUid=1000,OwnerGid=
```

Set up access point

Here's a brief explanation of the code:

- `--file-system-id`: It specifies the ID of the EFS file system that the access point will be created for.
- `--posix-user Uid=1000,Gid=1000`: It sets the default user and group ID that Lambda will use to access the file system. These values simulate a Linux user with UID and GID of `1000`, ensuring consistent permission enforcement.
- `--root-directory`: It defines the entry point in the file system and automatically creates the directory if it doesn't exist.
  - `Path=/lambda`: Lambda will mount the file system at the `/lambda` directory.
  - `CreationInfo={OwnerUid=1000,OwnerGid=1000,Permissions=750}`: It creates the `/lambda` directory with:
    - Owner UID and GID set to 1000.
    - Permissions `750` (owner: read/write/execute; group: read/execute; others: no access).

Ask

Step 3: Associate the Lambda function with EFS

Once the access point is configured, we can associate the Lambda function with EFS by using the `update-function-configuration` command:

main.sh Shell

```
aws lambda update-function-configuration \
  --function-name my-function \
  --file-system-configs Arn=arn:aws:elasticfilesystem:region:account-
  --vpc-config SubnetIds=subnet-1,subnet-2,SecurityGroupIds=<valid_se
```

Associate Lambda with EFS

Here's a brief explanation of the command:

- `--function-name`: It identifies the target Lambda function to update.
- `--file-system-configs`: It specifies the EFS access point ARN that the function should use to access the file system securely.
  - `LocalMountPath`: It defines where in the Lambda function's execution environment the EFS file system will be mounted. The function code will read/write to this path.
- `--vpc-config`: These are subnets in the same VPC as the EFS mount targets. Lambda randomly selects one during execution.
  - `SecurityGroupIds`: The security group must allow outbound tra port `2049` (NFS) to communicate with EFS mount targets.

Ask

Step 4: Use the EFS inside the Lambda code

Finally, the EFS is ready to be used in the Lambda function code as follows:

main.py Python

```python
def lambda_handler(event, context):
    with open("/mnt/efs/output.txt", "w") as file:
        file.write("Hello from Lambda using EFS!")

    with open("/mnt/efs/output.txt", "r") as file:
        content = file.read()

    return {"message": content}
```

Lambda function code to store data

This AWS Lambda function writes a message to a file stored in an Amazon EFS (Elastic File System) mounted at `/mnt/efs/`, then reads the content back from the file. It returns the file content as part of the response in JSON format.

Monitoring Amazon EFS with CloudWatch

Monitoring is a key part of developing resilient and high-performing applications, and Amazon EFS integrates natively with Amazon CloudWatch to provide insight into our file system's health and activity. EFS automatically emits metrics such as `BurstCreditBalance`, `ClientConnections`, `DataReadIOBytes`, and `DataWriteIOBytes`. These metrics help us understand how much throughput our application is consuming, whether we're at risk of depleting burst credits, and how many clients are connected to the file system.

Ask

For example, if we're using the default bursting throughput mode, monitoring the `BurstCreditBalance` can alert us before the file system enters a throttled state.

Similarly, if our workload suddenly increases and performance degrades, reviewing `PermittedThroughput` vs. `TotalIOBytes` can help identify a potential saturation issue.

Monitoring EFS

CloudWatch allows us to set alarms on any of these metrics. We can configure alerts to notify our team via Amazon SNS if usage exceeds expected thresholds or if read/write operations drop unexpectedly, indicating a possible application or connectivity issue. For Lambda integrations, while function-level logs are sent to CloudWatch Logs, performance of the underlying EFS access still relies on CloudWatch EFS metrics for visibility.

In addition to metrics, CloudTrail logs can be used to track management-level operations on EFS, such as the creation of file systems, mount targets, or access points. Together, these tools allow us to take a proactive approach for identifying latency, optimizing throughput, and ensuring that our application makes the most

Ask